

ATIVIDADE AVALIATIVA — Módulo Complementar

Verificação de Equivalência Formal (Formality)

RTL × Netlist do Controlador de Vending Machine, com guidance via SVF

Trabalho individual ou em duplas — pré-requisito: ter concluído a atividade "Controlador de Vending Machine em SystemVerilog", incluindo a síntese com Design Compiler (synth.tcl, vending.sdc e a netlist gerada).

⚠ Este roteiro cobre exclusivamente o fluxo de equivalência formal (Formality), incluindo o uso do arquivo SVF gerado pelo Design Compiler como guidance para o Formality. Não é necessário nenhum ambiente de property checking (VC Formal) para completá-lo.

1. Objetivo

Nesta atividade você irá fechar formalmente o sign-off da síntese do controlador da vending machine, provando que a netlist gerada pelo Design Compiler é sequencialmente equivalente ao RTL original, usando o Formality da Synopsys — desta vez usando o arquivo SVF (Setup/Verification File) gerado pelo próprio Design Compiler durante a síntese como guidance de casamento, em vez de depender só da análise estrutural do Formality. Ao final você será capaz de:

- Instrumentar synth.tcl para que o Design Compiler grave um arquivo SVF com o histórico de otimizações feitas durante compile_ultra;
- Carregar esse mesmo SVF no Formality como guidance, antes de ler os designs golden e revision;
- Executar match e verify e interpretar os vereditos SUCCEEDED, FAILED, NOT COMPARED e INCONCLUSIVE;
- Usar report_svf_operation para distinguir otimizações de síntese aceitas automaticamente como guidance das que precisam de investigação manual;
- Comparar o efeito da opção -no_autoungroup usada na síntese sobre a facilidade de casamento de pontos, com e sem guidance de SVF;
- Gerar um único trilho de auditoria (SVF) contínuo, da síntese até a equivalência formal, para anexar ao relatório técnico.

2. Por que equivalência formal, se a netlist já foi simulada?

A atividade de síntese pede simulação da netlist pós-síntese com os mesmos vetores do testbench de RTL — isso é chamado de gate-level simulation (GLS) e é útil, mas tem duas limitações estruturais: primeiro, só prova que a netlist se comporta como o RTL para os vetores que você rodou (os mesmos 4 cenários obrigatórios); segundo, GLS com atrasos unit-delay ou zero-delay não modela glitches e corridas (races) reais, e é lenta demais para rodar milhões de vetores.

A equivalência formal (equivalence checking) resolve isso de outra forma: em vez de simular vetores, o Formality prova matematicamente — via BDDs e SAT — que, para qualquer sequência de entradas possível, cada saída e cada elemento de estado (flip-flop) da netlist calcula exatamente o mesmo valor que o elemento correspondente no RTL golden, ciclo a ciclo. Isso cobre 100% do espaço de entradas, não apenas os cenários testados.

Regra prática: GLS prova que "a netlist passou nos testes que rodamos"; Formality prova que "a netlist é logicamente idêntica ao RTL, para qualquer entrada" — ou aponta exatamente onde e por que diverge.

3. Conceitos-chave

Termo	Significado
Golden design (referência)	O design que se assume correto — aqui, o RTL SystemVerilog original (vending_pkg.sv, credit_reg.sv, memory.sv, comparator.sv, subtractor.sv, control_unit.sv, vending_top.sv).
Revised design (implementação)	O design a ser verificado contra o golden — aqui, a netlist gerada por write -format verilog -hierarchy ao final do synth.tcl.
Compare points	Pontos de comparação: saídas primárias e, principalmente, cada flip-flop/latch do design. O Formality tenta casar (match) cada compare point do golden com um do revised antes de verificar a equivalência lógica entre eles.
match / verify	match casa pontos por nome, estrutura e vizinhança lógica; verify prova (via BDD/SAT) se a lógica combinacional entre cada par casado é equivalente.
SVF (Setup/Verification File)	Arquivo binário que registra, passo a passo, as transformações e decisões de uma sessão de ferramenta Synopsys. É o elemento central deste roteiro: o Design Compiler pode escrever um SVF durante compile_ultra registrando cada otimização (boundary optimization, retiming, merge/inversão de registrador, reencodificação de FSM); o Formality pode ler esse mesmo arquivo como guidance para casar pontos automaticamente, sem precisar redescobrir essas transformações por análise estrutural pura.

Nomes exatos de comando e opções podem variar ligeiramente conforme a versão instalada no laboratório. Use `dc_shell -help` e `fm_shell -help`, ou a documentação Synopsys (SolvNet) local, para confirmar a sintaxe exata da versão disponível — os scripts abaixo são o roteiro de referência, não um literal a ser copiado sem adaptação.

4. Estrutura de diretórios (para este roteiro)

```

grupo_NN_vending/
├── rtl/                                (da atividade anterior – golden)
│   ├── vending_pkg.sv
│   ├── credit_reg.sv
│   ├── memory.sv
│   ├── comparator.sv
│   ├── subtractor.sv
│   ├── control_unit.sv
│   └── vending_top.sv
├── synth/
│   ├── synth.tcl                      (atualizado nesta atividade – grava o SVF)
│   ├── vending.sdc
│   ├── vending_top_netlist.v          (revision – write -format verilog -hierarchy)
│   └── reports/
│       ├── default.svf                Guidance gerado pelo Design Compiler
│       ├── dwsvf_default/             (se houver componentes DesignWare mapeados)
│       └── ... (area/timing/power da síntese)
├── fm/
│   ├── formality.tcl                  Lê e estende o mesmo default.svf
│   ├── formality_submodules.tcl       (opcional) verificação hierárquica por bloco
│   └── reports/
│       ├── formality_status.rpt
│       ├── formality_passing.rpt
│       ├── formality_failing.rpt
│       ├── formality_unmatched.rpt
│       ├── formality_svf_accepted.rpt
│       └── formality_svf_rejected.rpt
└── relatorio_equivalencia.pdf

```

Note que existe um único arquivo `default.svf`, gerado pelo Design Compiler e reaberto pelo Formality — não dois arquivos separados. Essa continuidade é o que torna o SVF um trilho de auditoria único, da síntese até a prova de equivalência.

5. Etapa 1 — Gerando o guidance SVF no Design Compiler

Para que o Formality receba automaticamente o histórico de otimizações do Design Compiler, adicione `set_svf` ao `synth.tcl` da atividade anterior, imediatamente antes de `compile_ultra`. A partir desse ponto, toda decisão de otimização tomada durante a compilação (remoção de lógica redundante, propagação de constantes, retiming, merge/inversão de registradores e, potencialmente, reencodificação da FSM) fica registrada no arquivo.

```
# ---- synth.tcl (trecho) – adição para gerar o guidance SVF ----

# ... (target_library, link_library, analyze, elaborate, link,
#      read_sdc vending.sdc, check_design – como na atividade anterior)

# Grava o histórico de otimizações do Design Compiler a partir daqui.
# Precisa vir ANTES de compile_ultra para capturar toda a compilação.
set_svf reports/default.svf

compile_ultra -no_autoungroup

# ... (report_area, report_timing, report_power,
#      report_constraint -all_violators, write -format verilog -hierarchy
#      – como na atividade anterior)
```

Se o Design Compiler mapear o comparador ou o subtrator para uma célula DesignWare (ex.: DW01_cmp2 ou DW01_sub), ele também pode gerar uma pasta auxiliar (por exemplo `dwsvf_default/`) ao lado do `.svf`. Mantenha essa pasta junto do arquivo — o Formality a referencia internamente e ela não pode ser movida ou renomeada isoladamente.

6. Etapa 2 — Ambiente do Formality e leitura do SVF de guidance

No Formality, carregue a mesma biblioteca de células da síntese e, em seguida, aponte `set_svf` para o MESMO arquivo escrito pelo Design Compiler na etapa anterior. Como o arquivo já existe, o Formality o interpreta automaticamente como guidance — e continua registrando as próprias operações da sessão atual no mesmo arquivo, estendendo o trilho de auditoria.

```
# =====
# formality.tcl – Equivalência RTL (golden) x Netlist (revision)
# Controlador de Vending Machine – com guidance de SVF
# =====

# 1. Biblioteca de células – a mesma target_library do synth.tcl
read_db /caminho/da/biblioteca/typical.db

# 2. Carrega o SVF gerado pelo Design Compiler (Etapa 1) como guidance.
# Deve vir ANTES de qualquer read_verilog/set_top: o Formality lê
# o histórico de otimizações da síntese e passa a registrar também
# as próprias operações desta sessão no mesmo arquivo.
set_svf reports/default.svf

# 3. ESSENCIAL: habilita o modo de setup automático baseado no SVF.
# Sem esta variável, o Formality só aproveita o guidance
# parcialmente (limitado a poucas operações estruturais) – é ela
# que faz o Formality de fato consumir o histórico de otimizações
# do Design Compiler (constantes, retiming, merge/inversão de
# registrador, reencodificação de FSM) durante o match.
set synopsys_auto_setup true
```

Não pule esta variável: `set synopsys_auto_setup true` é o que conecta o guidance gravado no `default.svf` ao comportamento do `match/verify`. Sem ela, mesmo com o SVF carregado, o Formality tende a tratar boa parte das transformações do Design Compiler como estruturais desconhecidas — voltando a exigir o mesmo trabalho manual (`match -certain`, `set_user_match`) que o roteiro usa o SVF justamente para evitar.

Dica de depuração: o SVF é um formato binário. Se precisar inspecionar manualmente o que foi gravado (por exemplo, para conferir se o `compile_ultra` realmente rodou depois do `set_svf` em `synth.tcl`), gere uma versão texto com a função de `dump/print` do próprio Formality (consulte `fm_shell -help` por algo como "imprimir"/"dump" de SVF na sua versão) antes de prosseguir para o `match`.

6.1 Gerando o esqueleto do script automaticamente com `fm_mk_script`

Em vez de escrever `formality.tcl` inteiramente à mão como acima, o Formality traz um utilitário de linha de comando chamado `fm_mk_script`, que lê um ou mais arquivos SVF e gera automaticamente um script Tcl inicial de equivalência — com os comandos de leitura de biblioteca, leitura de design e `set_svf` inferidos a partir do que está registrado no SVF. Diferente de `match`, `verify` e `read_verilog`, `fm_mk_script` roda no shell do sistema operacional, fora de uma sessão do `fm_shell`.

```
# No shell do sistema (fora do fm_shell), a partir do diretório fm/:

# Uso básico: gera fm_mk_script.tcl a partir do SVF do Design Compiler
fm_mk_script ../synth/reports/default.svf

# Especificando o nome do script de saída
fm_mk_script ../synth/reports/default.svf -output formality_auto.tcl
```

O script gerado traz os comandos de `setup` (bibliotecas, `read_verilog`, `set_top`, `set_svf`) inferidos a partir dos nomes de arquivo e da hierarquia registrados no `default.svf`, mas normalmente ainda precisa de ajustes: caminhos relativos corretos para a estrutura de diretórios do grupo, e a sequência de `match` / `verify` / `report_*` que fecha o sign-off — `fm_mk_script` gera o `setup` de leitura, não a verificação completa.

Use como segundo par de olhos: gere `formality_auto.tcl` a partir do mesmo `default.svf` usado na Etapa 2 e compare, linha a linha, com o `formality.tcl` escrito manualmente nesta seção. Reconcilie qualquer divergência (por exemplo, um arquivo RTL que o SVF não registrou por não ter sofrido nenhuma otimização, ou um caminho de biblioteca diferente) antes de rodar `match/verify` — divergências aqui costumam indicar um erro de digitação no script manual, não um bug do design. Documente no relatório se você usou `fm_mk_script` como ponto de partida ou apenas como conferência do script manual.

7. Etapa 3 — Design de referência (golden = RTL)

Leia o mesmo conjunto de arquivos RTL entregue na atividade anterior, na mesma ordem de dependência usada em `analyze` no Design Compiler (pacote primeiro, módulos folha depois, `top` por último). Aponte `vending_top` como `top` do design golden.

```
# 4. Design de referência (golden) – RTL pré-síntese
read_verilog -r { ../rtl/vending_pkg.sv \
                  ../rtl/credit_reg.sv \
                  ../rtl/memory.sv \
                  ../rtl/comparator.sv \
                  ../rtl/subtractor.sv \
                  ../rtl/control_unit.sv \
                  ../rtl/vending_top.sv }
set_top r:/WORK/vending_top
```

Se o RTL usa SystemVerilog puro (`typedef enum`, `packages`), confirme com `read_verilog -help` se sua versão precisa de uma flag adicional (ex.: `-sv02012` ou `-sverilog`) para interpretar corretamente `vending_pkg.sv` — caso contrário o Formality pode falhar já na leitura, antes mesmo de chegar ao `match`.

8. Etapa 4 — Design revisado (revision = netlist)

Leia a netlist gerada por `write -format verilog -hierarchy` ao final de `synth.tcl`. Aponte `vending_top` como `top` do design revised.

```
# 5. Design revisado – netlist gerada pelo Design Compiler
read_verilog -i ../synth/vending_top_netlist.v
set_top i:/WORK/vending_top
```

Se o Design Compiler tiver instanciado células DesignWare (ex.: DW01_sub no subtrator) ou primitivas de biblioteca não cobertas pelo .db carregado, o Formality vai reportar essas instâncias como black boxes, mesmo com o guidance do SVF presente. Nesse caso é preciso carregar também a biblioteca/model correspondente antes do match.

9. Etapa 5 — match: casamento de pontos guiado por SVF

Com o guidance do SVF já carregado (Etapa 2), o comando match aplica automaticamente boa parte das transformações que o Design Compiler registrou, além da análise estrutural padrão (nome, posição e assinatura lógica da vizinhança). Rode:

```
# 6. Casamento de pontos entre golden e revised, usando o guidance do SVF
match

# Verifica quais operações do SVF foram usadas com sucesso como guidance
# e quais foram rejeitadas (precisam de investigação manual)
report_svf_operation -status accepted > reports/formality_svf_accepted.rpt
report_svf_operation -status rejected > reports/formality_svf_rejected.rpt

# Inspeção o resultado do casamento antes de seguir para verify
report_matched_points > reports/formality_matched.rpt
report_unmatched_points > reports/formality_unmatched.rpt
```

9.1 Causas típicas de unmatched points neste design (com guidance de SVF)

Causa	Onde costuma aparecer no controlador	Como resolver, agora com o SVF de guidance
Reencodificação de estado da FSM	control_unit.sv tem 6 estados codificados em binário puro (3 bits); o Design Compiler pode reencodar para one-hot ou gray durante compile_ultra	Com set_svf carregado na Etapa 2, essa reencodificação já vem registrada no guidance e normalmente é resolvida automaticamente no match. Confirme em report_svf_operation -status accepted; se aparecer como rejected, só então recorra a set_user_match manual ou fixe a codificação no Design Compiler antes de resintetizar
Propagação de constantes / remoção de lógica redundante	Bits do memory.sv que nunca mudam de valor (ex.: bit de paridade não usado) podem ser otimizados e desaparecer da netlist	Também costuma vir coberto pelo guidance; confira em report_svf_operation -status accepted. Só investigue manualmente se aparecer em -status rejected
Retiming / balanceamento de registradores	Pouco provável neste design (não há retiming habilitado no synth.tcl da atividade anterior), mas pode ocorrer se -no_autoungroup for removido e o DC juntar hierarquias	O guidance do SVF é justamente recomendado pela Synopsys para esses casos; se ainda assim algo ficar unmatched, gere um novo SVF a partir de uma nova rodada de compile_ultra e repita a Etapa 1/2

Causa	Onde costuma aparecer no controlador	Como resolver, agora com o SVF de guidance
Nomes de sinal/hierarquia diferentes após ungroup	Se compile_ultra for rodado sem -no_autoungroup, módulos como comparator e subtractor podem ser fundidos (ungroup automático) na netlist, mudando nomes hierárquicos	O guidance do SVF cobre a maior parte dessas mudanças; para o restante, rode match -certain como segunda passada antes de declarar unmatched

Pontos ainda unmatched após consultar o guidance não impedem necessariamente o verify — o Formality reporta o restante do design como NOT COMPARED, mas isso deve ser investigado e justificado no relatório técnico, nunca ignorado silenciosamente.

10. Etapa 6 — verify: prova de equivalência

```
# 7. Prova de equivalência ponto a ponto
verify

# 8. Relatórios de sign-off
report_status -verbose      > reports/formality_status.rpt
report_passing_points      > reports/formality_passing.rpt
report_failing_points      > reports/formality_failing.rpt
report_unmatched_points    > reports/formality_unmatched.rpt

exit
```

Veredito	Significado	Ação esperada
SUCCEEDED	Todos os pontos casados foram provados equivalentes, e não há pontos NOT COMPARED relevantes não justificados	Sign-off aprovado — anexar formality_status.rpt e o par accepted/rejected do SVF ao relatório técnico
FAILED	Ao menos um par de pontos casados é logicamente diferente	Analisar report_failing_points; ver seção 10.1 para causas típicas neste design
INCONCLUSIVE	O solver não conseguiu provar nem refutar a equivalência de um ou mais pontos dentro dos limites de esforço padrão	Tentar estratégias alternativas de prova (ver seção 10.2) antes de escalar como bug
NOT COMPARED	Existem pontos golden ou revised sem correspondente, mesmo após o guidance do SVF (ver seção 9.1)	Investigar cada unmatched point restante; não é automaticamente um bug, mas exige justificativa

10.1 Failing points — causas reais de bug a investigar

- Biblioteca (.db) carregada é diferente da usada pelo Design Compiler na síntese — a causa mais comum de falso failing point em ambiente de laboratório;
- O RTL lido pelo Formality não é exatamente o mesmo RTL usado no analyze/elaborate do Design Compiler ao gerar o default.svf (ex.: RTL editado depois da síntese, sem ressintetizar);
- O arquivo default.svf usado é de uma rodada de compile_ultra diferente da netlist que está sendo lida como revision (ex.: SVF de uma execução antiga, netlist de uma execução mais recente) — sempre regenere os dois juntos;
- Uma constraint em vending.sdc (ex.: um false_path mal aplicado) mudou o comportamento funcional em vez de apenas orientar timing;
- Erro real de lógica introduzido por uma diretiva de otimização agressiva — exatamente o cenário que este roteiro existe para pegar antes do tape-out.

10.2 Quando o resultado fica INCONCLUSIVE

Para um design pequeno como este controlador (6 estados de FSM, datapath de 8 bits), inconclusivos são raros mesmo sem guidance, e ainda mais raros com o SVF carregado. Se ocorrerem em algum ponto específico (por exemplo, no comparador ou subtrator), experimente:

- Confirmar que o guidance do SVF foi de fato aplicado a esse ponto (report_svf_operation -status accepted);
- Aumentar o esforço de prova sobre o datapath (opções de effort mais altas específicas para comparadores/subtratores, disponíveis conforme a versão instalada);
- Isolar o cone de lógica do ponto problemático e verificá-lo separadamente (verificação hierárquica, seção 12);
- Consultar analyze_points para entender qual entrada do cone lógico não foi resolvida.

11. Etapa 7 — Efeito de -no_autoungroup na equivalência (com e sem SVF)

A atividade de síntese pede compile_ultra -no_autoungroup como opção principal, e depois pede para repetir a síntese sem essa opção (Parte 3, item 5) para comparar área e timing. Repita a verificação de equivalência para as duas netlists, cada uma com o SEU PRÓPRIO SVF de guidance — um SVF gravado durante uma rodada de compile_ultra não deve ser reaproveitado para verificar a netlist da outra rodada, pois cada rodada faz otimizações diferentes.

1. Rode synth.tcl com -no_autoungroup, gerando vending_top_netlist_grouped.v e reports/default_grouped.svf; rode formality.tcl apontando para esse par e registre o status;
2. Rode synth.tcl sem -no_autoungroup, gerando vending_top_netlist_ungrouped.v e reports/default_ungrouped.svf; rode formality.tcl novamente, trocando as duas linhas (set_svf e read_verilog -i) para o novo par, e registre o status;
3. Compare, entre as duas rodadas, o número de compare points, unmatched points, e quantas operações do SVF foram accepted vs. rejected;
4. No relatório, explique: hierarquia preservada (-no_autoungroup) tende a gerar um SVF mais simples e um casamento de pontos mais direto; ungroup automático tende a gerar mais transformações registradas no SVF (e, ainda assim, o guidance deve reduzir bastante o trabalho manual de match - certain comparado a rodar sem SVF algum).

12. Etapa 8 (opcional, nível avançado) — Verificação hierárquica por submódulo

Como o projeto já separa claramente datapath e controle em arquivos distintos (credit_reg.sv, memory.sv, comparator.sv, subtractor.sv, control_unit.sv), é possível — e recomendado como exercício extra — verificar cada submódulo isoladamente antes de verificar o top-level, reaproveitando o mesmo SVF gerado para o design completo.

```
# formality_submodules.tcl – exemplo para o control_unit isoladamente
read_db /caminho/da/biblioteca/typical.db
set_svf ../synth/reports/default.svf

read_verilog -r { ../rtl/vending_pkg.sv ../rtl/control_unit.sv }
set_top r:/WORK/control_unit

read_verilog -i ../synth/vending_top_netlist.v
set_top i:/WORK/vending_top/control_unit_i

match
verify
report_status -verbose > reports/control_unit_status.rpt
exit
```


Repita o padrão acima para `credit_reg`, `memory`, `comparator` e `subtractor`, ajustando o caminho hierárquico do `set_top -i` para a instância correspondente dentro de `vending_top_netlist.v`. Esse é o mesmo tipo de decomposição hierárquica sugerido para casos de INCONCLUSIVE na seção 10.2.

13. Tabela de referência rápida de comandos

Comando	Uso
<code>set_svf (dc_shell)</code>	No Design Compiler, colocado ANTES de <code>compile_ultra</code> : grava em arquivo o histórico de otimizações da síntese, para uso posterior como guidance
<code>fm_mk_script</code> (shell do sistema)	Utilitário de linha de comando (fora do <code>fm_shell</code>) que lê um ou mais arquivos SVF e gera automaticamente um script Tcl inicial de setup do Formality — útil como esqueleto ou como conferência do script escrito à mão
<code>set_svf (fm_shell)</code>	No Formality, colocado ANTES de <code>read_verilog/set_top</code> : se o arquivo já existe (foi escrito pelo DC), é lido como guidance; a sessão atual continua sendo registrada no mesmo arquivo
<code>set synopsys_auto_setup true</code>	ESSENCIAL junto com <code>set_svf</code> : sem esta variável o guidance do SVF é usado apenas parcialmente. É ela que faz o Formality efetivamente consumir o histórico de otimizações do Design Compiler no <code>match/verify</code>
<code>read_db</code>	Carrega bibliotecas de células (.db) — deve ser idêntica à <code>target_library</code> da síntese
<code>read_verilog -r / -i</code>	Lê o design de referência (golden, -r) ou revisado (revision, -i)
<code>set_top r:/... / i:/...</code>	Define o módulo top de cada design (prefixo r: para golden, i: para revision)
<code>match</code>	Casa compare points entre golden e revised, usando o guidance do SVF quando disponível
<code>match -certain</code>	Segunda passada de casamento mais agressiva, útil para o que restar unmatched após o guidance
<code>set_user_match</code>	Casamento manual de um ponto específico, reservado para o que o guidance do SVF não resolveu
<code>verify</code>	Prova a equivalência lógica de cada par de pontos casados
<code>report_svf_operation -status accepted / rejected</code>	Lista quais operações do guidance do SVF foram aplicadas com sucesso e quais foram rejeitadas — o principal indicador de quanto trabalho manual ainda resta
<code>report_status -verbose</code>	Resumo geral do sign-off (SUCCEEDED / FAILED / INCONCLUSIVE)
<code>report_matched_points / report_unmatched_points</code>	Lista detalhada de pontos casados / não casados
<code>report_passing_points / report_failing_points</code>	Lista detalhada de pontos provados equivalentes / divergentes
<code>analyze_points</code>	Investiga em detalhe o cone lógico de um ponto específico (útil para failing/inconclusive)

14. Critérios de sign-off desta etapa

Critério	Comando de verificação	Condição de aprovação
----------	------------------------	-----------------------

Critério	Comando de verificação	Condição de aprovação
SVF de guidance gerado junto com a síntese	presença de reports/default.svf após synth.tcl	Arquivo existe e corresponde à mesma rodada de compile_ultra que gerou a netlist usada
Setup automático do SVF habilitado no Formality	set synopsys_auto_setup true presente em formality.tcl, antes do match	Variável definida logo após o set_svf — sem ela o guidance só é parcialmente aproveitado
Guidance do SVF aplicado no Formality	report_svf_operation -status accepted/rejected	Maioria das operações registradas aparece como accepted; toda rejected está justificada
Netlist principal equivalente ao RTL golden	report_status -verbose	SUCCEEDED, sem failing points
Unmatched points remanescentes justificados	report_unmatched_points	Cada ponto ainda unmatched após o guidance tem causa identificada e documentada
Comparação -no_autoungroup vs. sem essa opção	duas rodadas completas (synth.tcl + SVF + formality.tcl)	Ambas chegam a SUCCEEDED (ou têm divergência explicada) com seus respectivos SVFs

15. Relatório técnico (complemento ao relatório da atividade de síntese)

Adicione ao relatório entregue na atividade de síntese uma seção "Equivalência Formal (Formality) com guidance de SVF" contendo:

1. Trecho do synth.tcl mostrando onde set_svf foi inserido e confirmação de que reports/default.svf foi gerado;
2. Trecho do formality.tcl mostrando set_svf seguido de set synopsys_auto_setup true, antes de qualquer read_verilog;
3. Print de report_svf_operation -status accepted e -status rejected, com explicação de qualquer operação rejeitada;
4. Print de report_status -verbose mostrando o veredito final para a netlist principal (com -no_autoungroup);
5. Lista de unmatched points remanescentes (se houver) com a causa identificada para cada um, usando a tabela da seção 9.1 como referência;
6. Comparação lado a lado dos resultados de equivalência com e sem -no_autoungroup (seção 11), incluindo número de compare points, operações accepted/rejected do SVF e tempo de execução;
7. Se algum failing point real for encontrado: trace do cone lógico via analyze_points, causa-raiz e correção proposta (no RTL, no script de síntese ou no SDC);
8. Discussão: quanto trabalho manual de matching o guidance do SVF eliminou, comparado ao fluxo sem SVF descrito informalmente em aula.

16. Critérios de avaliação

Item de avaliação	Pontos	Critério
-------------------	--------	----------

Item de avaliação	Pontos	Critério
Geração do SVF de guidance na síntese	15	synth.tcl atualizado com set_svf antes de compile_ultra; reports/default.svf gerado e correspondente à netlist entregue
Ambiente Formality configurado corretamente	10	formality.tcl roda sem erros fatais; biblioteca, SVF e designs golden/revision carregados na ordem correta; set synopsys_auto_setup true presente logo após o set_svf
Match e uso do guidance do SVF	20	match executado; report_svf_operation analisado; unmatched points remanescentes investigados e justificados
Verify e sign-off da netlist principal	20	verify executado; report_status documentado; SUCCEEDED ou failing points corretamente investigados
Comparação -no_autoungroup vs. sem a opção	15	Duas rodadas completas (com seus próprios SVFs) comparadas, com discussão da diferença no casamento de pontos
Relatório técnico	20	Seção de equivalência formal completa, clara, com evidências (reports/prints) e o trilha de SVF documentado

Penalização: -10 pontos se o veredito final (report_status) não for reportado; -10 pontos se o SVF não for gerado pela síntese e usado pelo Formality (isto é, se o roteiro for seguido apenas com matching estrutural, sem guidance); -5 pontos se set synopsys_auto_setup true não estiver presente em formality.tcl; -5 pontos se nenhum unmatched/failing point remanescente for investigado quando existir algum.

Referências consultadas para os fluxos e nomenclatura das ferramentas:

- Synopsys — Formality Equivalence Checking (datasheet e página de produto)
- Synopsys — Formality and Formality Ultra datasheet
- Formality® Automated Setup File (SVF) Manual
- Formality User Guide (documentação de referência de comandos)